

Instruction Set Architecture I

Michael C. Hackett

Associate Professor, Computer Science



Lecture Topics

- Instruction Set Architectures
- Instruction Formats
 - Register Format Instructions
 - Immediate Format Instructions
 - Jump Format Instructions
- Pseudo Operations

Instruction Set Architectures

- An **instruction set architecture (ISA)** the overall design of a CPU's instructions.
- An ISA must be selected (or created) prior to designing a CPU
 - We can't design a CPU until we decide how we will tell it to add, subtract, and perform dozens of kinds other operations.

Instruction Formats

- An **instruction format** defines the arrangement and organization of the bits that form each instruction.
 - Each instruction is 32 bits in MIPS (same as its word size)
- This binary version of instructions is called the **machine language**
 - The assembler converts assembly instructions to machine language
 - The machine language is the actual instructions that are given to and understood by the CPU

Instruction Formats

- MIPS has three instruction formats:

- **Register Format (R Format)**

- Two operands and a target

add \$t2, \$t0, \$t1

- **Immediate Format (I Format)**

- Instructions with an immediate operand

addi \$t1, \$t0, 7

- **Jump Format (J Format)**

- Jump and Jump-and-Link instructions

j label

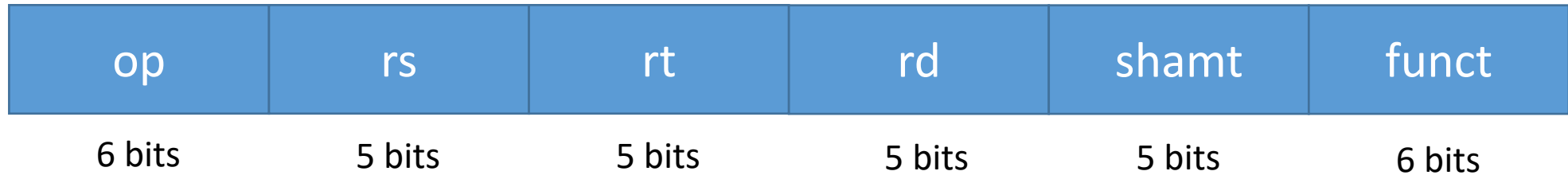
Instruction Formats

- See the MIPS Reference Card posted in Canvas as a reference for this lecture

MIPS Reference Data					ARITHMETIC CORE INSTRUCTION SET				
CORE INSTRUCTION SET				OPCODE / FUNCT (Hex)					OPCODE / FMT / FT / FUNCT (Hex)
NAME, MNEMONIC	FOR-MAT	OPERATION (in Verilog)			NAME, MNEMONIC	FOR-MAT	OPERATION		
Add	add	R	$R[rd] = R[rs] + R[rt]$	(1) 0 / 20 _{hex}	Branch On FP True	bclt	FI	$if(FPcond)PC=PC+4+BranchAddr$	(4) 11/8/1/-
Add Immediate	addi	I	$R[rt] = R[rs] + SignExtImm$	(1,2) 8 _{hex}	Branch On FP False	bclt	FI	$if(!FPcond)PC=PC+4+BranchAddr$	(4) 11/8/0/-
Add Imm. Unsigned	addiu	I	$R[rt] = R[rs] + SignExtImm$	(2) 9 _{hex}	Divide	div	R	$Lo=R[rs]/R[rt]; Hi=R[rs]\%R[rt]$	0/-/-/1a
Add Unsigned	addu	R	$R[rd] = R[rs] + R[rt]$	0 / 21 _{hex}	Divide Unsigned	divu	R	$Lo=R[rs]/R[rt]; Hi=R[rs]\%R[rt]$	(6) 0/-/-/1b
And	and	R	$R[rd] = R[rs] \& R[rt]$	0 / 24 _{hex}	FP Add Single	add.s	FR	$F[fd] = F[fs] + F[ft]$	11/10/-/0
And Immediate	andi	I	$R[rt] = R[rs] \& ZeroExtImm$	(3) c _{hex}	FP Add Double	add.d	FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} + \{F[ft], F[ft+1]\}$	11/11/-/0
Branch On Equal	beq	I	$if(R[rs]==R[rt])$ $PC=PC+4+BranchAddr$	(4) 4 _{hex}	FP Compare Single	c.x.s*	FR	$FPcond = (F[fs] op F[ft]) ? 1 : 0$	11/10/-/y
Branch On Not Equal	bne	I	$if(R[rs]!=R[rt])$ $PC=PC+4+BranchAddr$	(4) 5 _{hex}	FP Compare Double	c.x.d*	FR	$FPcond = (\{F[fs], F[fs+1]\} op \{F[ft], F[ft+1]\}) ? 1 : 0$ * (x is eq, lt, or le) (op is ==, <, or <=) (y is 32, 3c, or 3e)	11/11/-/y
Jump	j	J	$PC=JumpAddr$	(5) 2 _{hex}	FP Divide Single	div.s	FR	$F[fd] = F[fs] / F[ft]$	11/10/-/3
Jump And Link	jal	J	$R[31]=PC+8; PC=JumpAddr$	(5) 3 _{hex}	FP Divide Double	div.d	FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} / \{F[ft], F[ft+1]\}$	11/11/-/3
Jump Register	jtr	R	$PC=R[rs]$	0 / 08 _{hex}	FP Multiply Single	mul.s	FR	$F[fd] = F[fs] * F[ft]$	11/10/-/2
Load Byte Unsigned	lbu	I	$R[rt] = \{24'b0, M[R[rs] + SignExtImm][7:0]\}$	(2) 24 _{hex}	FP Multiply Double	mul.d	FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} * \{F[ft], F[ft+1]\}$	11/11/-/2
					FP Subtract Single	sub.s	FR	$F[fd] = F[fs] - F[ft]$	11/10/-/1
					FP Subtract Double	sub.d	FR	$\{F[fd], F[fd+1]\} = \{F[fs], F[fs+1]\} - \{F[ft], F[ft+1]\}$	11/11/-/1
					Load FP Single	lwc1	I	$F[rt] = M[R[rs] + SignExtImm]$	(2) 31/-/-/1

Register Format Instructions

- MIPS Register Format Instruction



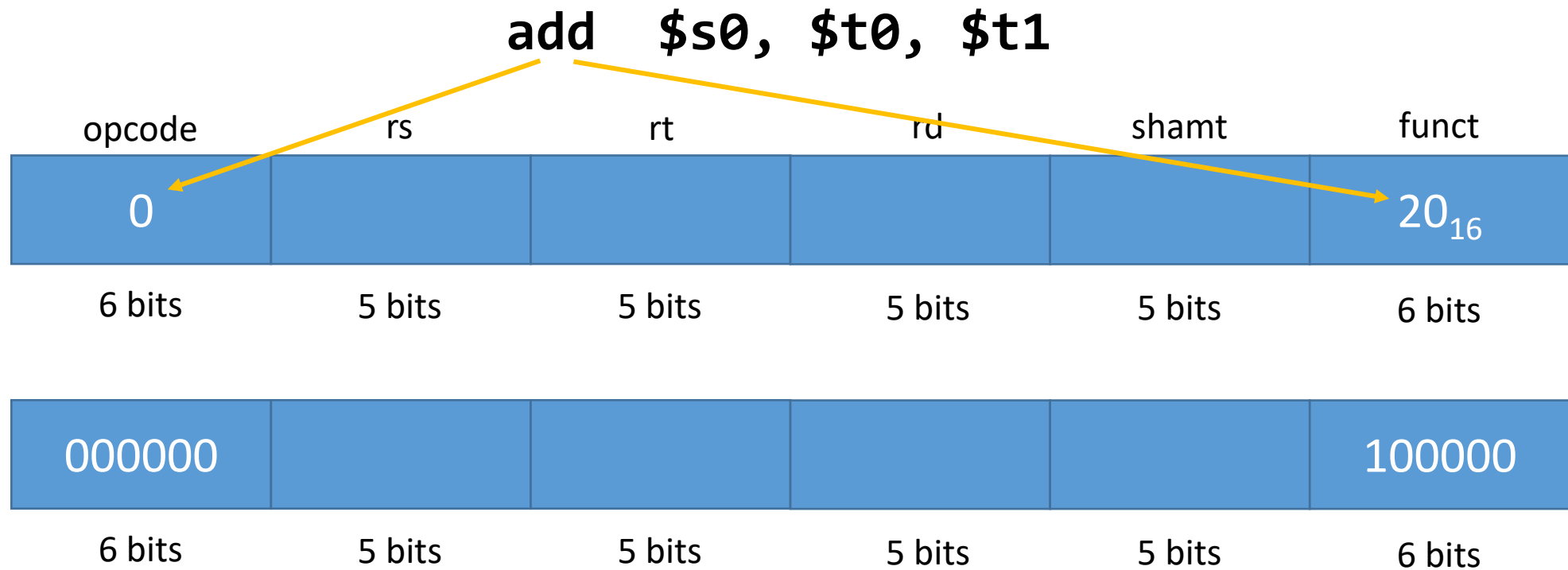
- op – Operation Code (**opcode**)
- rs – First source register (left operand)
- rt – Second source register (right operand)
- rd – Destination register
- shamt – Shift amount (used in shift operations)
- funct – Function code (specifies a variant of the opcode)

Register Format Instructions

- We see on the MIPS Reference Card, the **add** mnemonic has an opcode of 0 and a function code of 20_{16}

CORE INSTRUCTION SET			OPCODE / FUNCT (Hex)
NAME, MNEMONIC	FOR- MAT	OPERATION (in Verilog)	
Add	add	R $R[rd] = R[rs] + R[rt]$	(1) 0 / 20_{hex}

Register Format Instructions



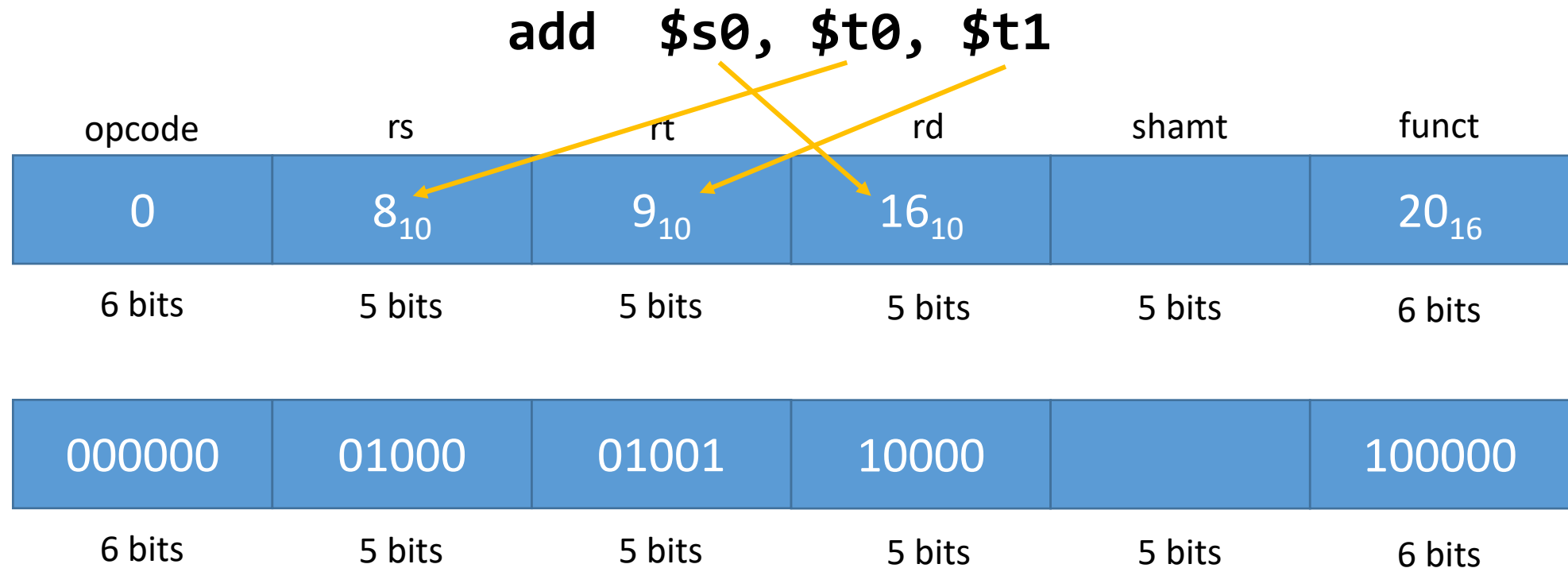
Register Format Instructions

- We can also see on the MIPS Reference Card, that each register corresponds to a number

REGISTER NAME, NUMBER, USE, CALL CONVENTION			
NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

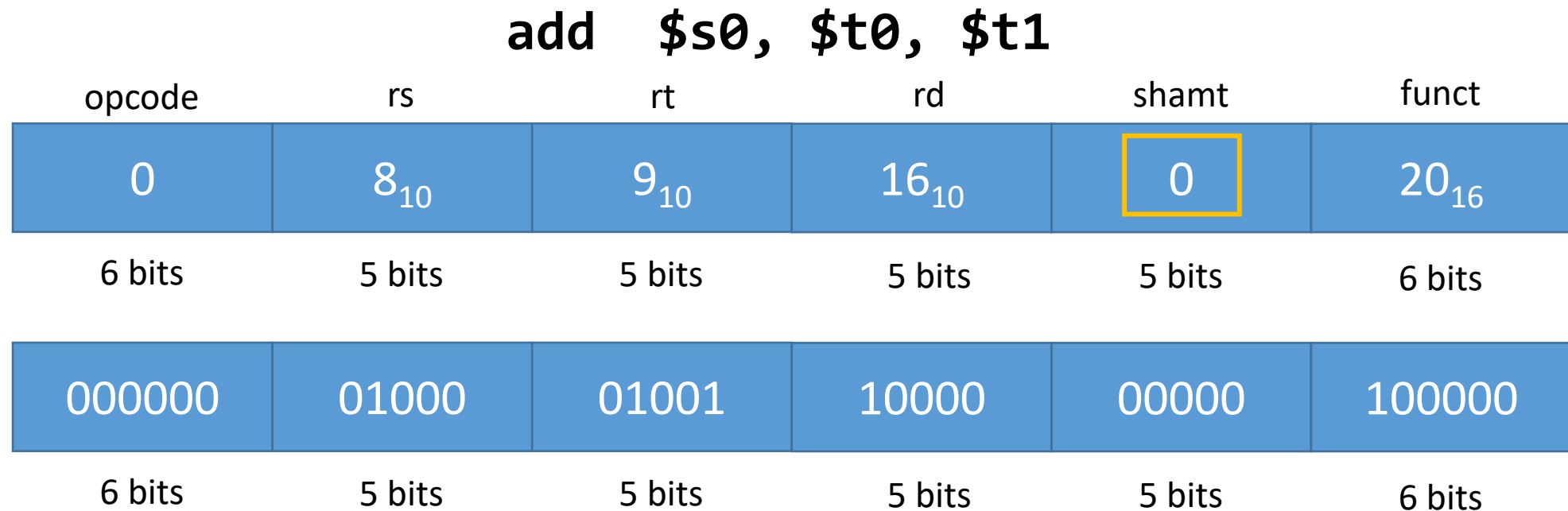
Register Format Instructions

\$t0-\$t7	8-15	Temporaries
\$s0-\$s7	16-23	Saved Temporaries

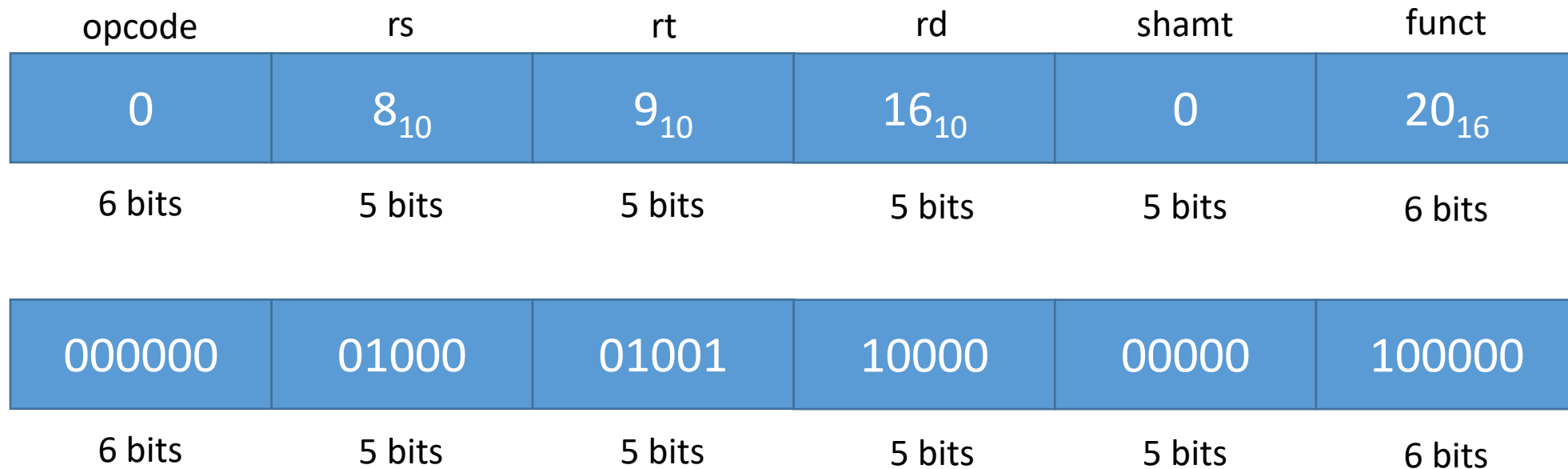


Register Format Instructions

- The **add** instruction does not perform a shift operation and therefore does not utilize the shamt field

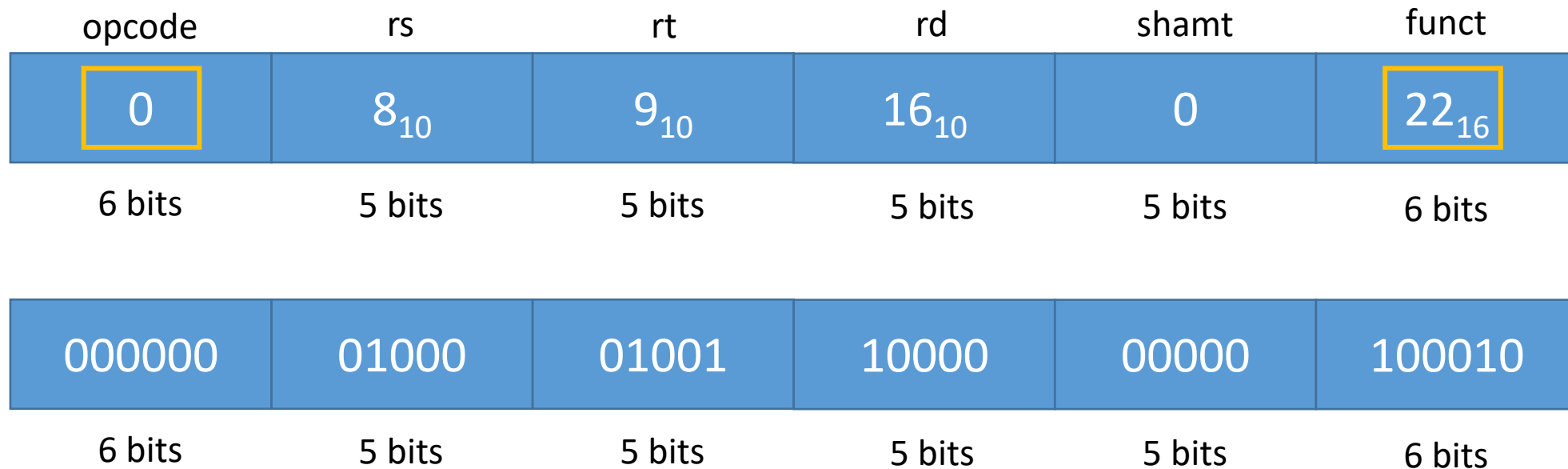


Register Format Instructions



- Assembly Language: **add \$s0, \$t0, \$t1**
- Machine Language: **0000 0001 0000 1001 1000 0000 0010 0000**

Register Format Instructions



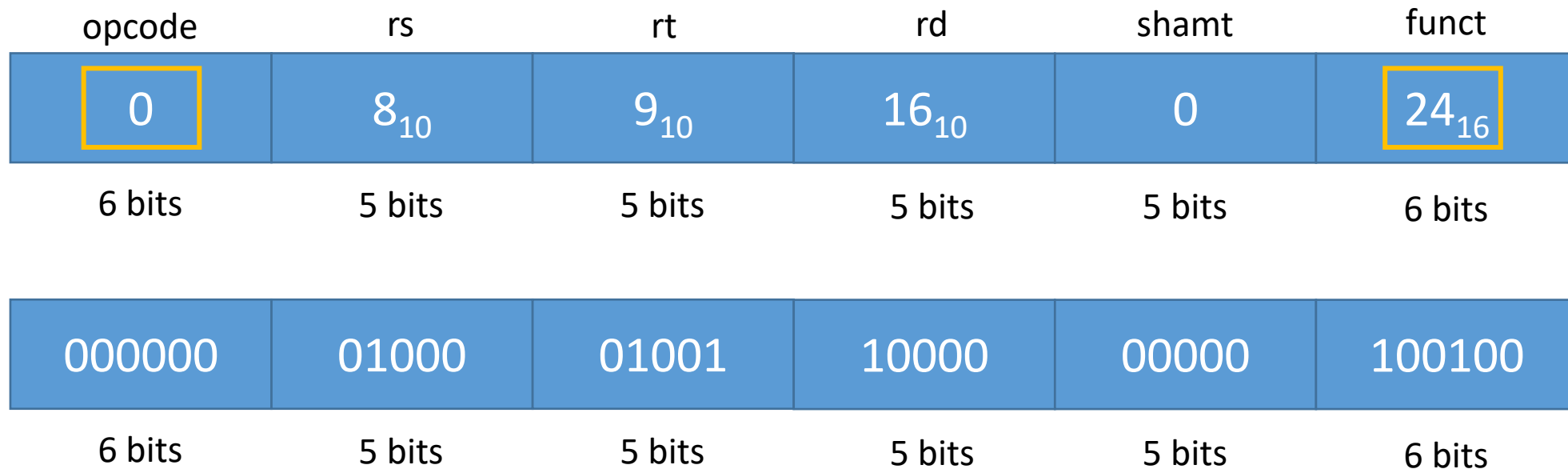
- Assembly Language:

sub \$s0, \$t0, \$t1

- Machine Language:

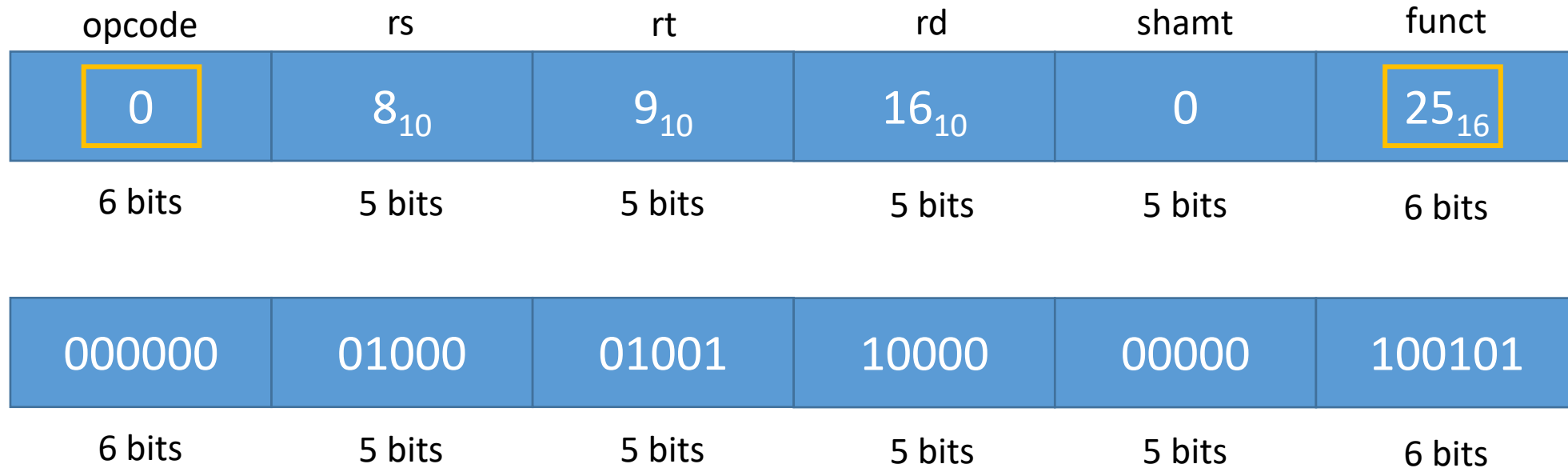
0000 0001 0000 1001 1000 0000 0010 0010

Register Format Instructions



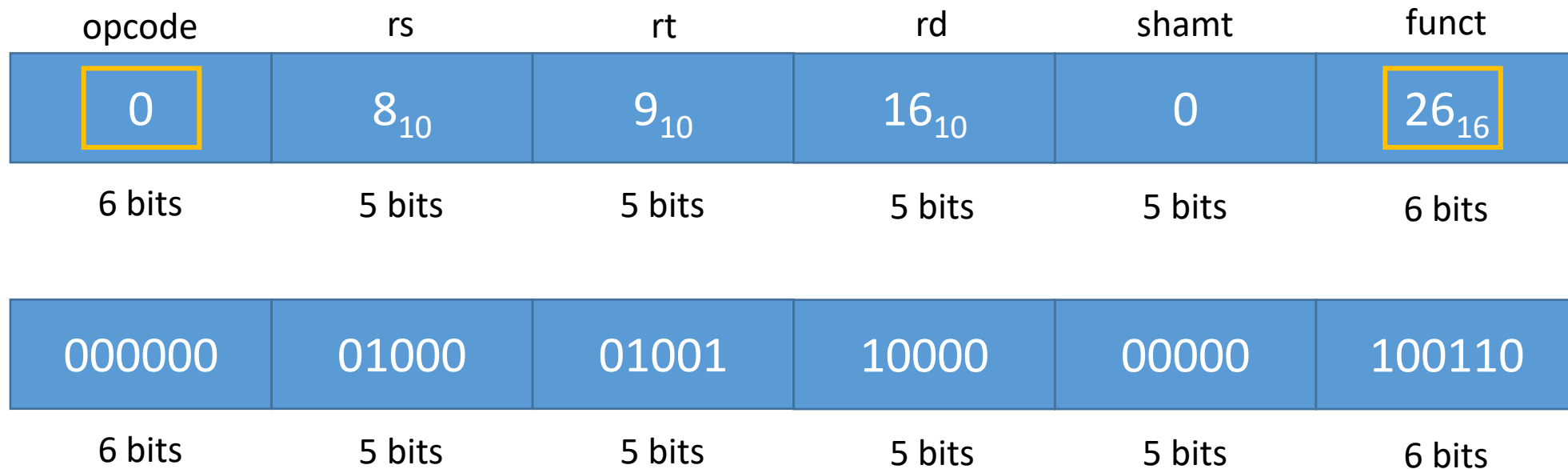
- Assembly Language: **and** \$s0, \$t0, \$t1
- Machine Language: 0000 0001 0000 1001 1000 0000 0010 0100

Register Format Instructions



- Assembly Language: **or** \$s0, \$t0, \$t1
- Machine Language: 0000 0001 0000 1001 1000 0000 0010 0101

Register Format Instructions



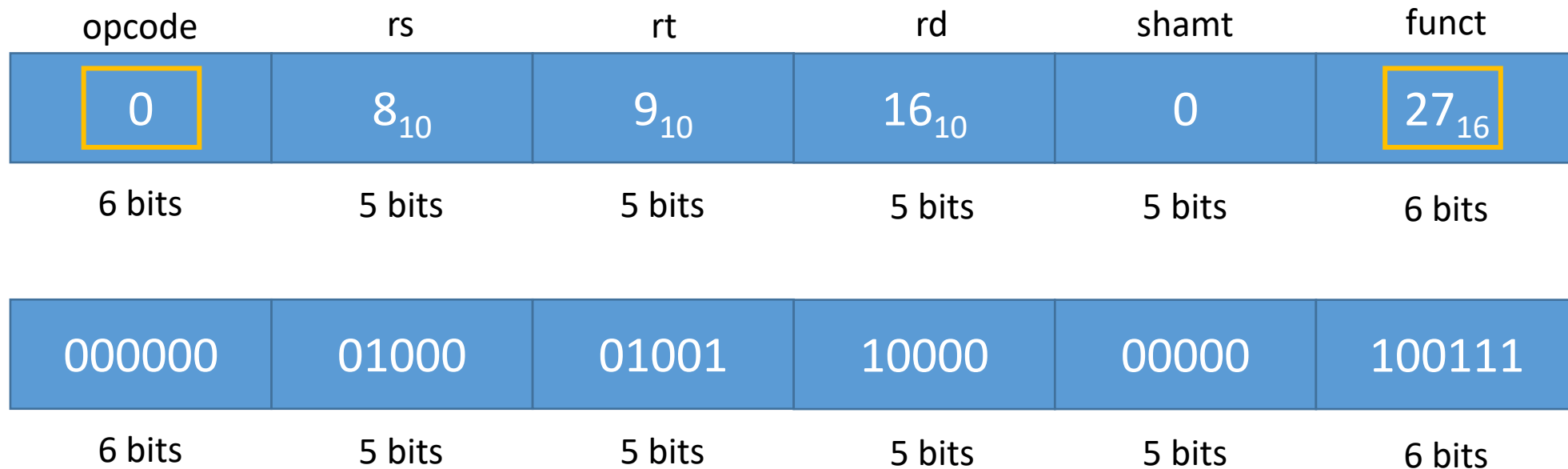
• Assembly Language:

xor \$s0, \$t0, \$t1

• Machine Language:

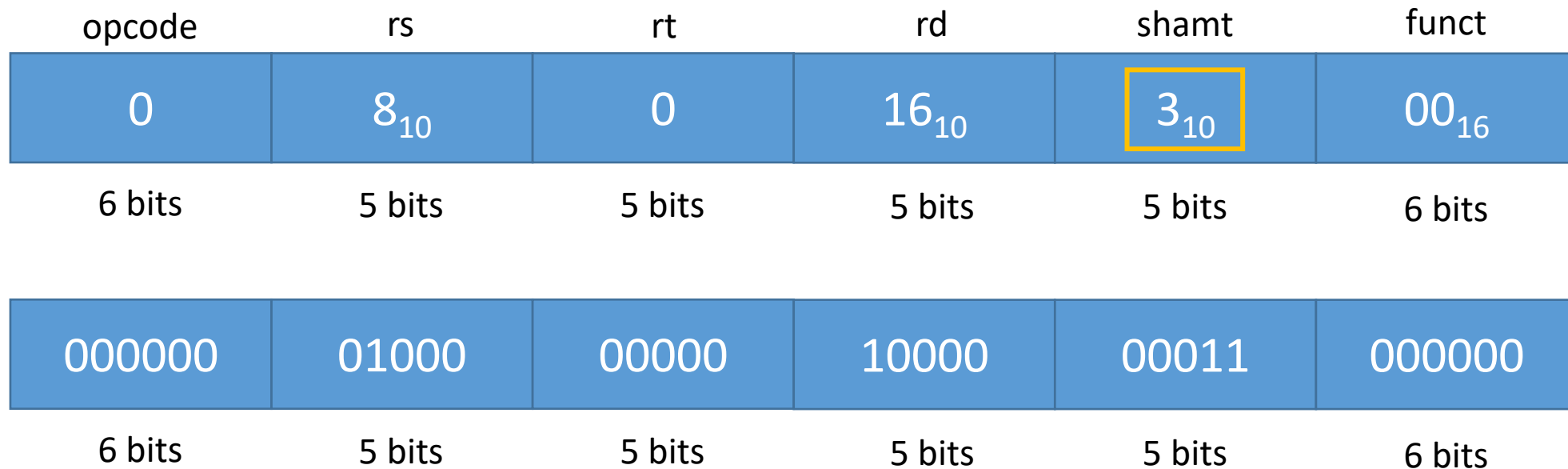
0000 0001 0000 1001 1000 0000 0010 0110

Register Format Instructions



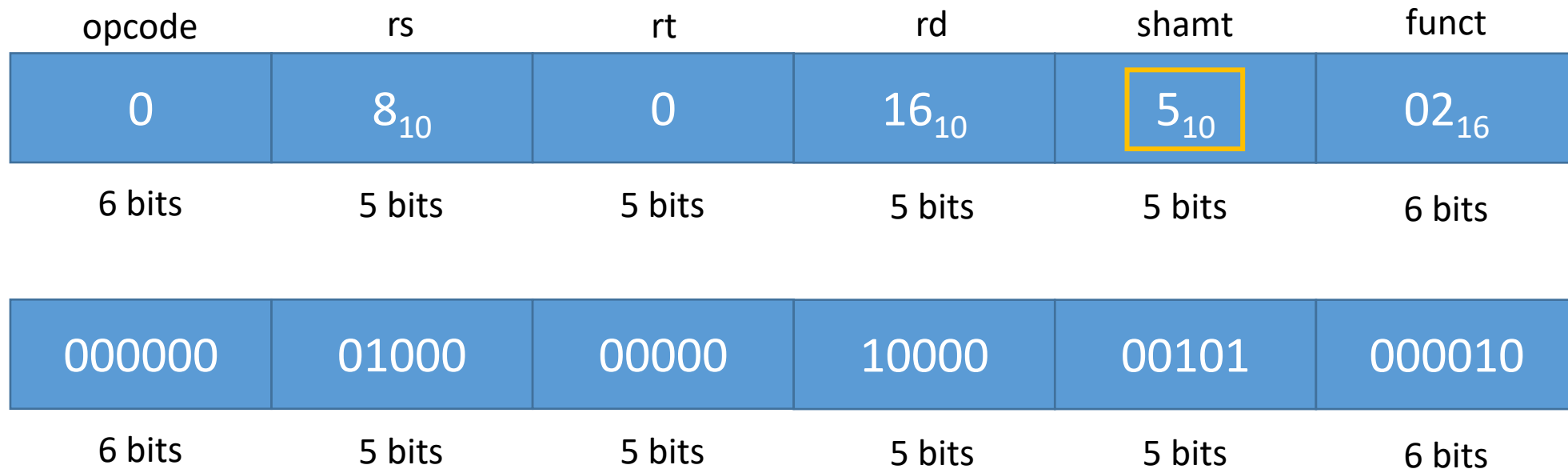
- Assembly Language: **nor** \$s0, \$t0, \$t1
- Machine Language: 0000 0001 0000 1001 1000 0000 0010 0111

Register Format Instructions



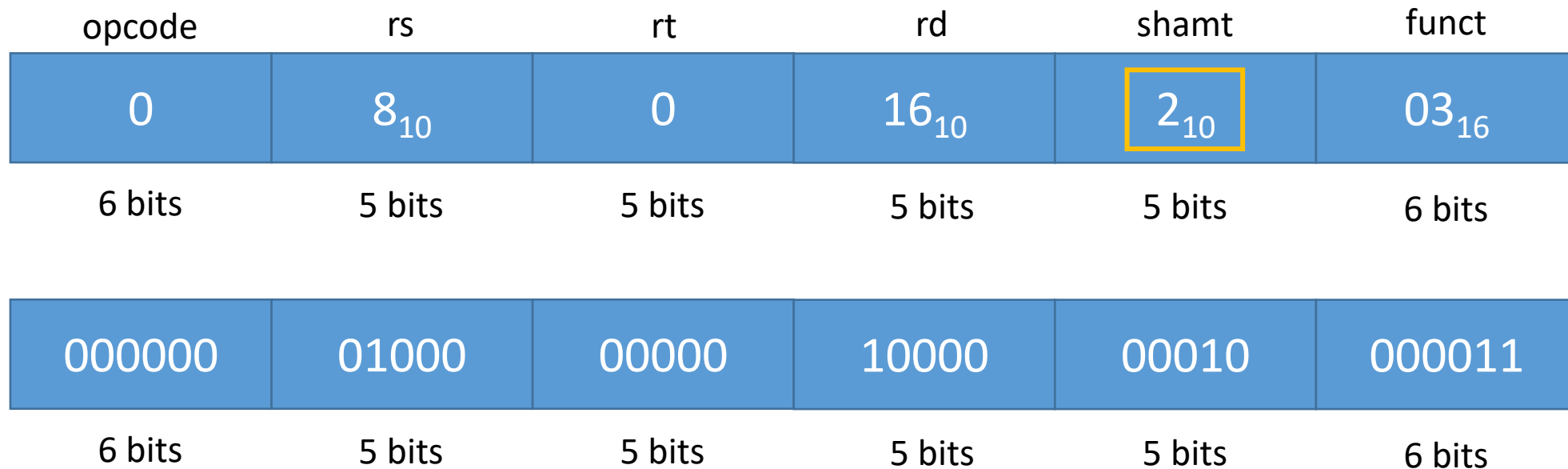
- Assembly Language: **sll \$s0, \$t0, 3**
- Machine Language: **0000 0001 0000 0000 1000 0000 1100 0000**

Register Format Instructions



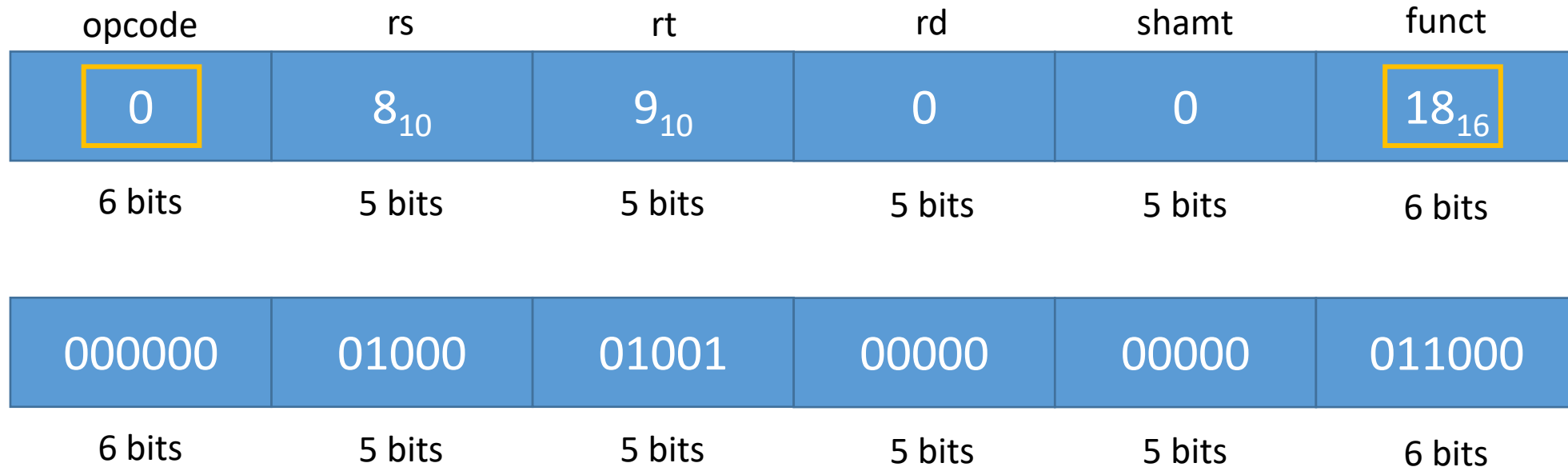
- Assembly Language: **sr1 \$s0, \$t0, 5**
- Machine Language: **0000 0001 0000 0000 1000 0000 1100 0010**

Register Format Instructions



- Assembly Language: **sra \$s0, \$t0, 2**
- Machine Language: **0000 0001 0000 0000 1000 0000 1100 0011**

Register Format Instructions



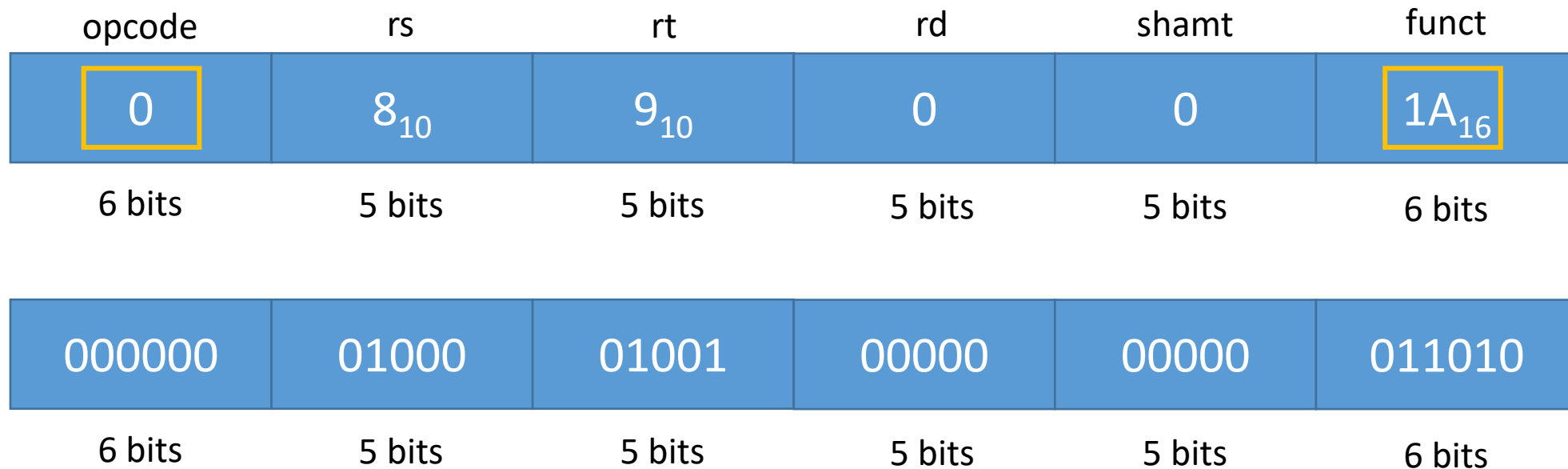
• Assembly Language:

mult \$t0, \$t1

• Machine Language:

0000 0001 0000 1001 0000 0000 0001 1000

Register Format Instructions



• Assembly Language:

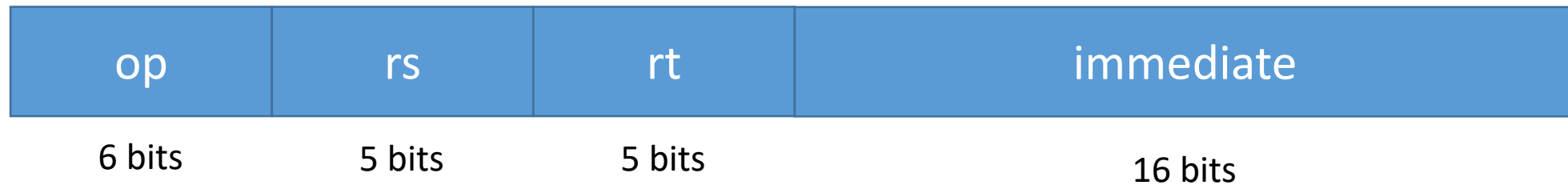
div \$t0, \$t1

• Machine Language:

0000 0001 0000 1001 0000 0000 0001 1010

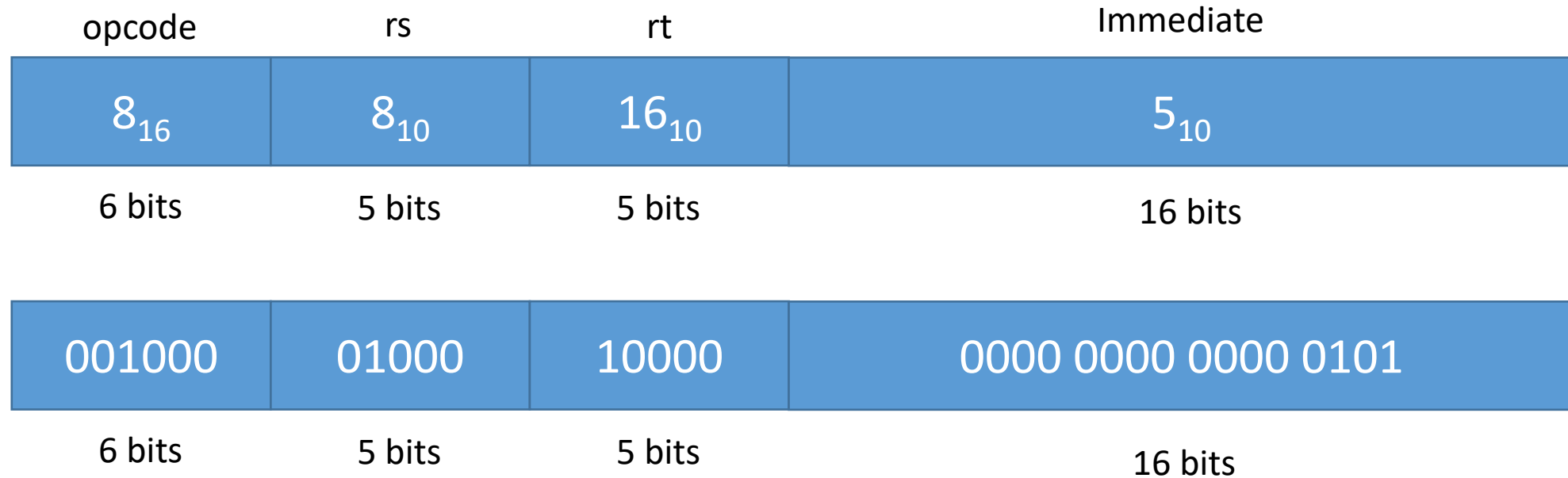
Immediate Format Instructions

- MIPS Immediate Format Instruction



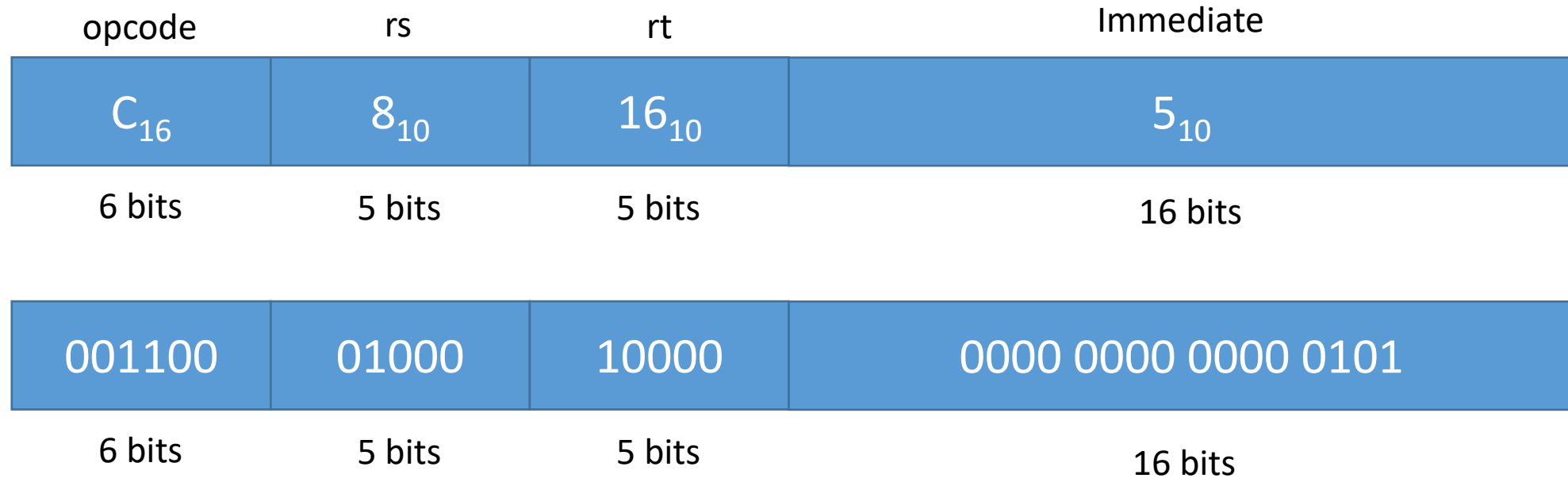
- op – Operation Code (opcode)
- rs – Source register
- rt – Second/Destination register

Immediate Format Instructions



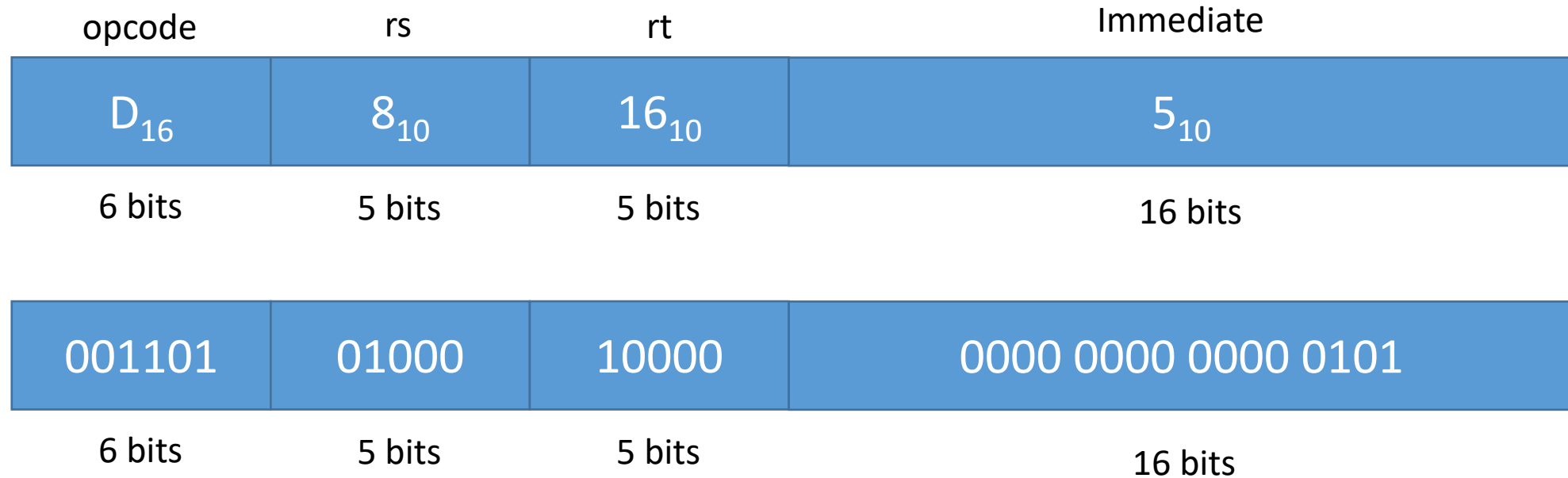
- Assembly Language: **addi \$s0, \$t0, 5**
- Machine Language: **0010 0001 0001 0000 0000 0000 0000 0101**

Immediate Format Instructions



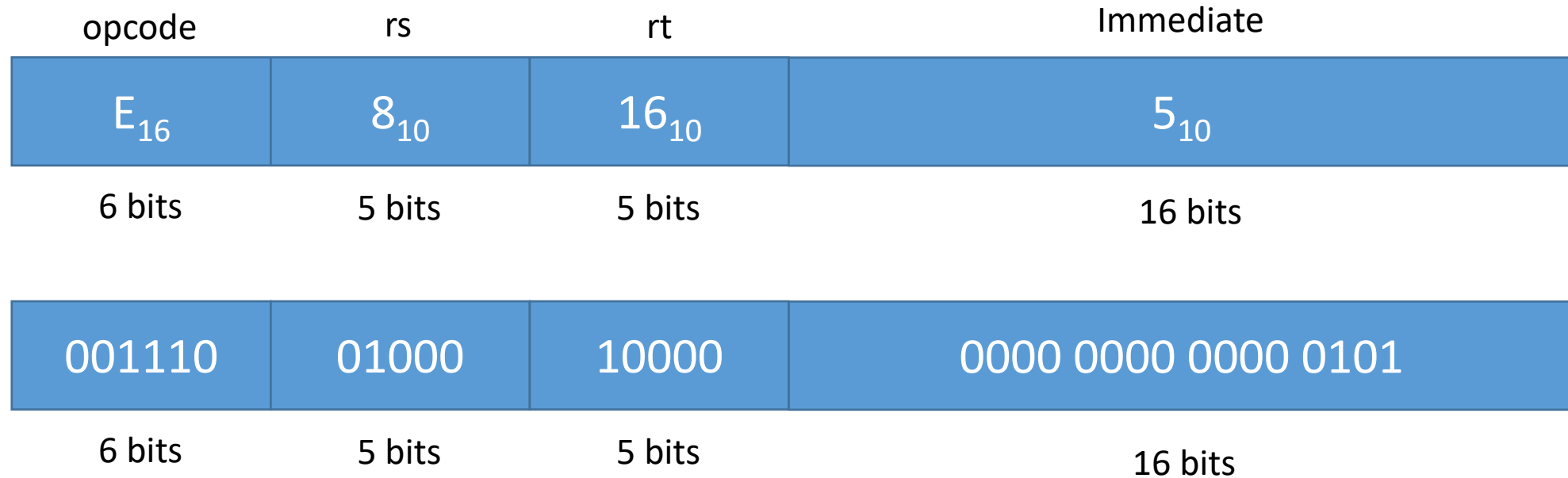
- Assembly Language: **andi \$s0, \$t0, 5**
- Machine Language: **0011 0001 0001 0000 0000 0000 0000 0101**

Immediate Format Instructions



- Assembly Language: **ori \$s0, \$t0, 5**
- Machine Language: **0011 0101 0001 0000 0000 0000 0000 0101**

Immediate Format Instructions

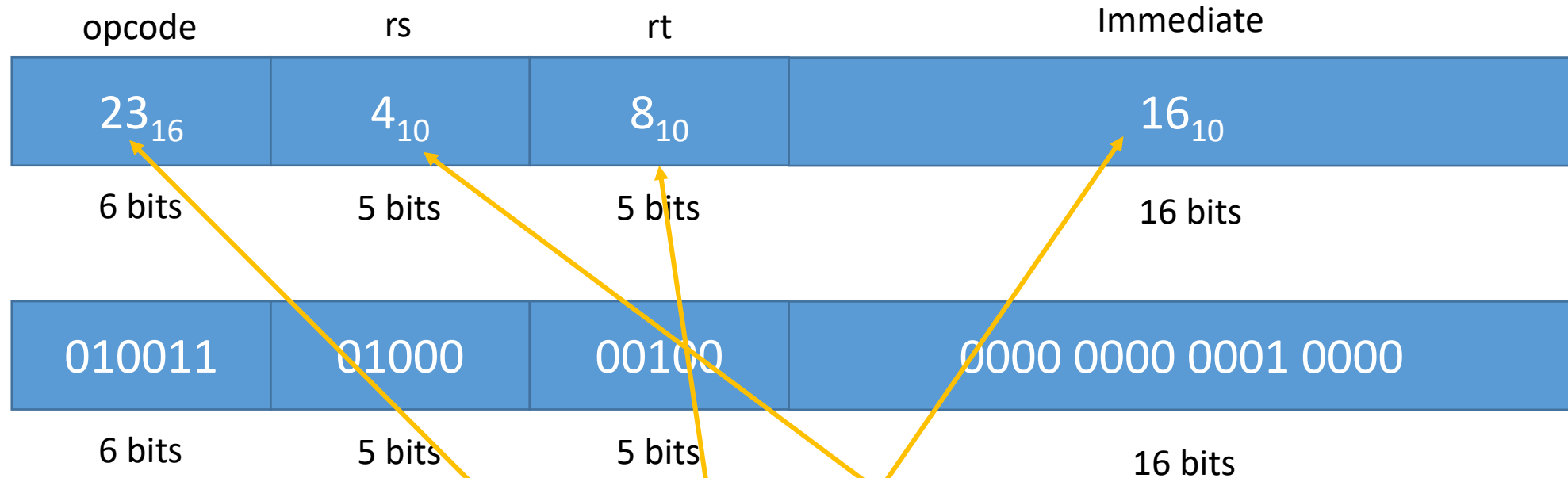


- Assembly Language: **xori \$s0, \$t0, 5**
- Machine Language: **0011 1101 0001 0000 0000 0000 0000 0101**

Immediate Format Instructions

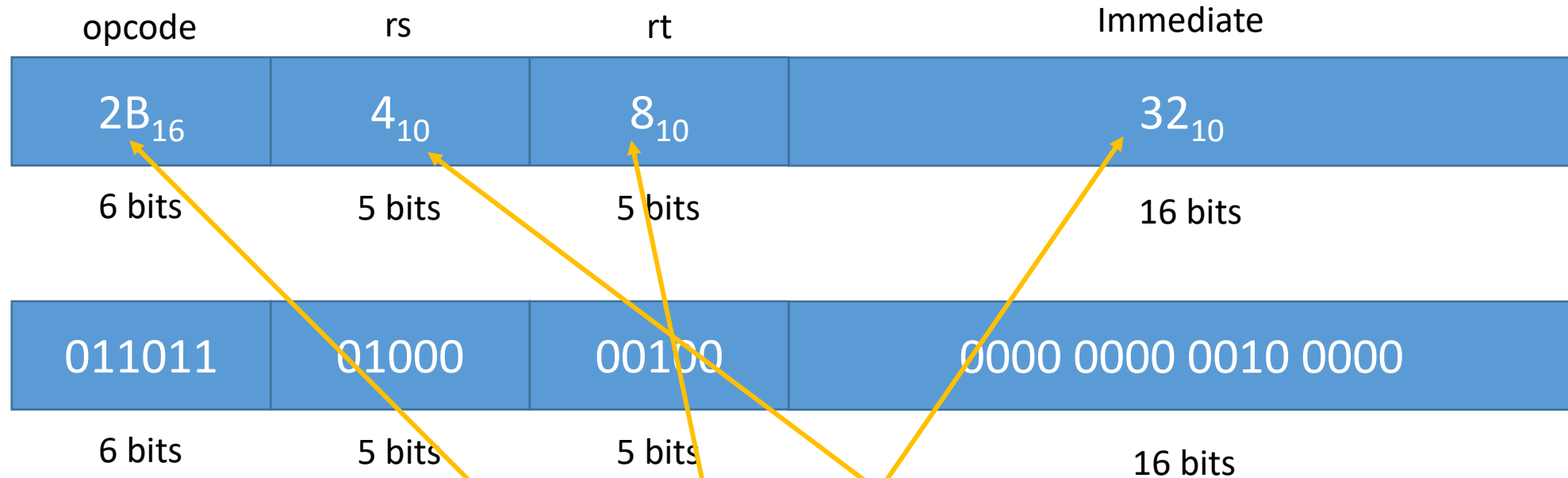
- Recall that memory reference instructions (which are I Format instructions) are used to move data to and from main memory.
- The instructions (**lw** and **sw**) were used for both non-symbolic and symbolic memory reference instructions.
 - Non-symbolic: **lw \$s0, 16(\$a0)**
 - Symbolic: **lw \$s0, mem_label**

Immediate Format Instructions



- Assembly Language: **lw \$t0, 16(\$a0)**
- Machine Language: **0100 1101 0001 0000 0000 0000 0001 0000**

Immediate Format Instructions



- Assembly Language: **sw \$t0, 32(\$a0)**
- Machine Language: **0110 1101 0001 0000 0000 0000 0001 0000**

Immediate Format Instructions

- When using a symbolic memory reference, a MIPS assembler will split a single **lw** or **sw** instruction into two instructions.
- Recall that the memory area in MIPS begins with the address 0x10010000
 - Each label in the data section is an offset from this starting address

Immediate Format Instructions

- We'll use the following data section of a hypothetical program as the example for the next two slides
 - Each word is 4 bytes (32-bits)

.data

value1	.word 55	#Starts at memory address 0x10010000 (offset 0)
value2	.word 106	#Starts at memory address 0x10010004 (offset 4)
value3	.word 99	#Starts at memory address 0x10010008 (offset 8)

Immediate Format Instructions

lw \$s0, value2

- First, we load the starting memory address to a register.
 - Here, we're using the \$at (assembler temporary) register

lui \$at, 0x1001

- This stores 0x10010000 to \$at

- Then, we load the data using the label's offset
 - value2 has an offset of 4

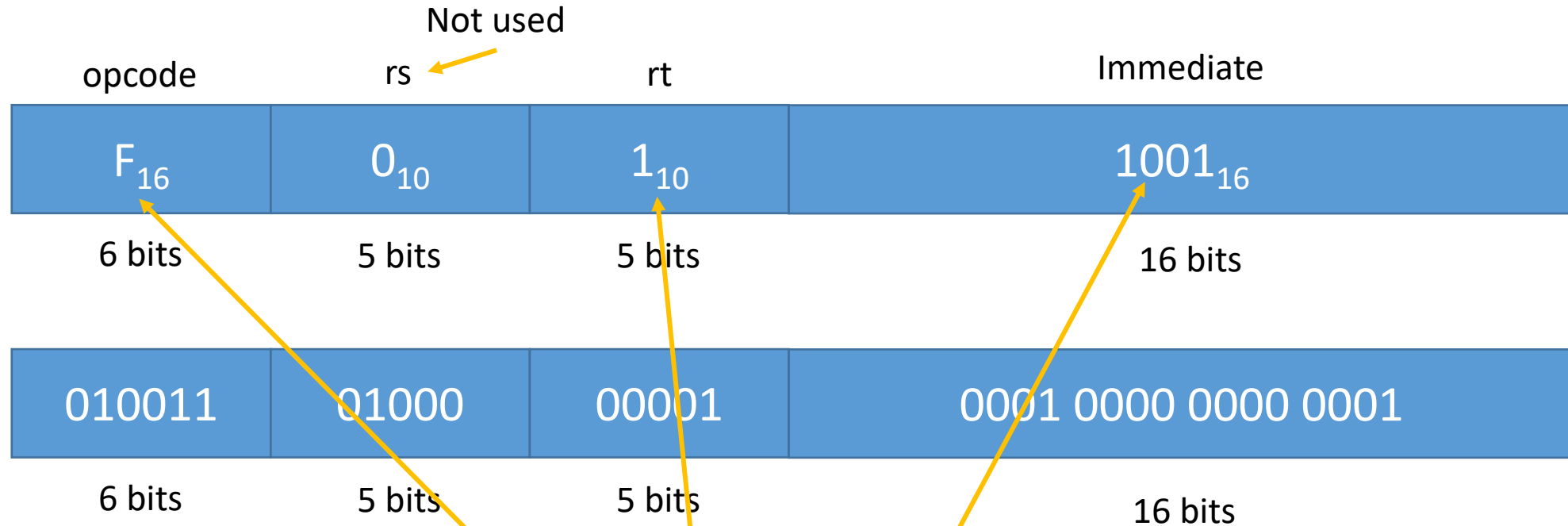
lw \$s0, 4(\$at)

lw \$s0, value2



lui \$at, 0x1001
lw \$s0, 4(\$at)

Immediate Format Instructions



- Assembly Language: **lui \$at, 0x1001**
- Machine Language: **0011 1100 0000 0001 0001 0000 0000 0001**

Immediate Format Instructions

- We previously saw six branching instructions (**beq**, **bgt**, etc.)
- Only **beq** and **bne** are actual MIPS instructions
 - The other four are pseudo-operations (discussed later in the lecture)
- In a branch instruction, we specified the label where the program should branch to.
 - In reality, we are moving X number of instructions forward or backward.

Immediate Format Instructions

.text

a:

li \$t0, 5

li \$t1, 6

bne \$t0, \$t1, b

li \$t2, 7

li \$t3, 8

b:

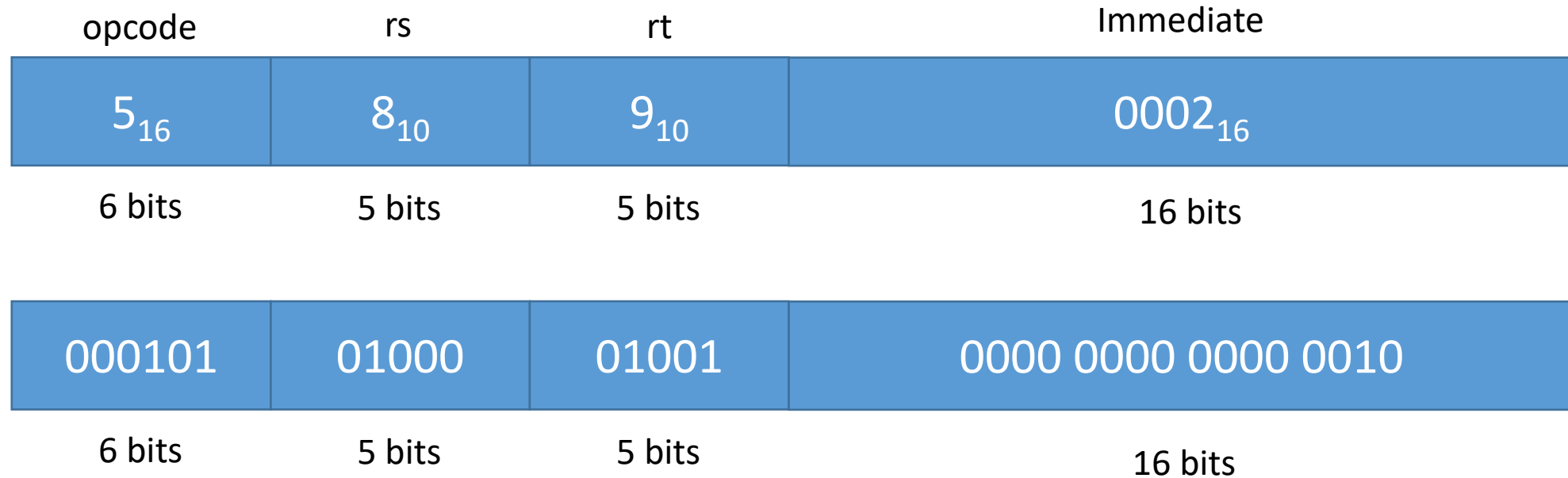
li \$t4, 9

li \$t5, 10

0
1
2

- This branch's condition is true, so the program advances to label b.
- In this program, we advanced two instructions.
 - This begins *after* the instruction following the branch
 - It helps to simply start counting from zero

Immediate Format Instructions



- Assembly Language: **bne \$t0, \$t1, *Label/offset***
- Machine Language: **0001 0101 0000 1001 0000 0000 0000 0010**

Immediate Format Instructions

.text

a:

li \$t0, 5 4

li \$t1, 6 3

b:

li \$t4, 9 2

li \$t5, 9 1

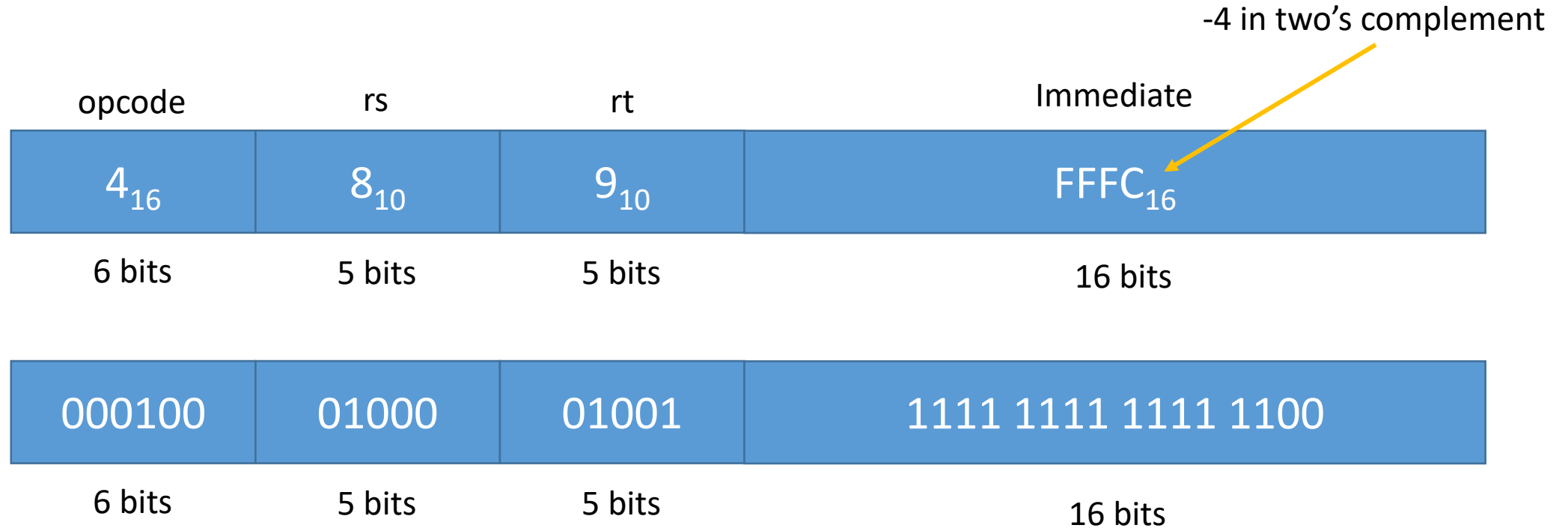
beq \$t0, \$t1, a 0

li \$t2, 7

li \$t3, 8

- This branch's condition is true, so the program goes back to label a.
- In this program, we went back four instructions.
 - This begins *starting with* the instruction preceding the branch instruction
- The offset is -4

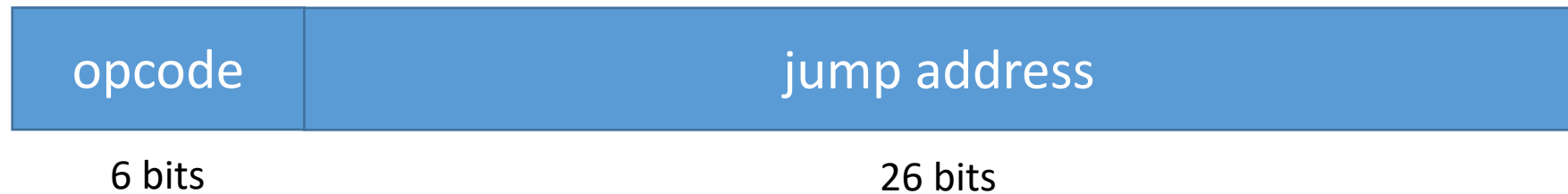
Immediate Format Instructions



- Assembly Language: **beq \$t0, \$t1, *Label/offset***
- Machine Language: **0001 0001 0000 1001 1111 1111 1111 1100**

Jump Format Instructions

- MIPS Jump Format Instruction



- op – Operation Code (opcode)
- MIPS only has two Jump Format Instructions: **j** and **jal**

Jump Format Instructions

.text

a:

li \$t0, 5

li \$t1, 6

j b

li \$t2, 7

li \$t3, 8

b:

li \$t4, 9

li \$t5, 10

- Unlike the relative addresses (offsets) used by branch instructions, jump instructions use absolute addresses.
- We'll say the address of the instruction labeled by b is 0x0040001C
- This is a 32-bit address, which won't fit in the 26-bit jump address portion of a J Format Instruction

Jump Format Instructions

- An instruction address will always end with 00_{16} , so we can omit those eight bits.
 - Memory is byte addressable and every instruction is 1 word/4 bytes/32-bits

0x0040001C

0000 0000 0100 0000 0000 0000 0001 1100

0000 0000 0100 0000 0000 0000 0001 1100

0000 0000 0100 0000 0000 0000 0001 1100

00 0001 0000 0000 0000 0000 0111

0x0100007

Jump Format Instructions

- Can be achieved one of two ways:
 1. Divide the address by 4 and take the lower 26 bits of the result
 - $0x0040001C / 4_{10} = 0x00100007 = \underline{0x010007}$
 2. Right shift the address by two bits, take the lower 26 bits of the result
 - Similar to what we did on the last slide

0x0040001C

0000 0000 0100 0000 0000 0000 0001 1100

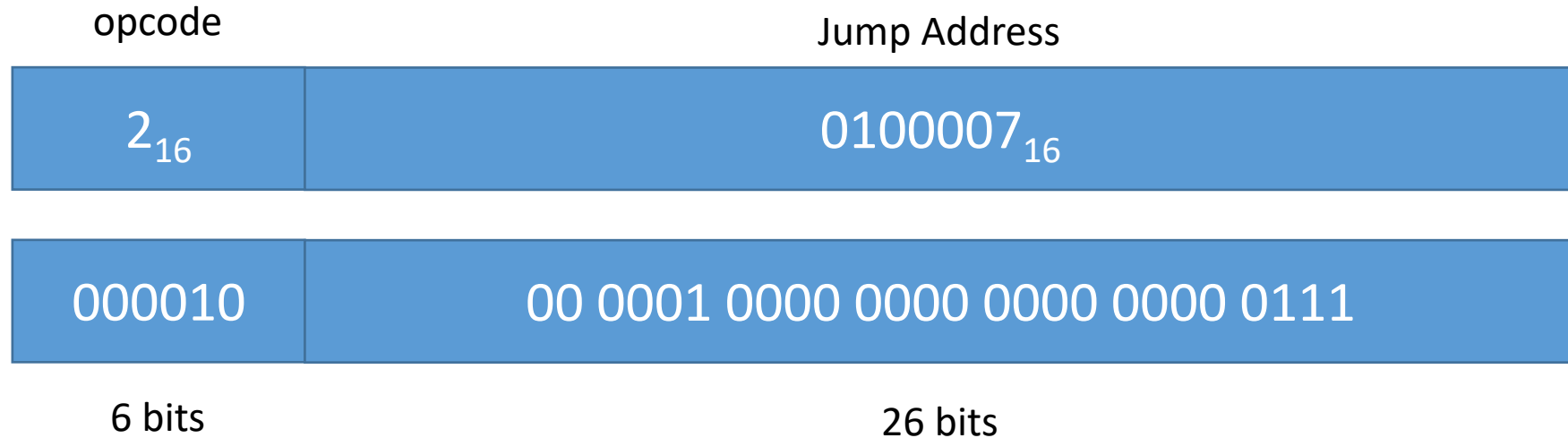
0000 0000 0001 0000 0000 0000 0000 0111

0000 0000 0001 0000 0000 0000 0000 0111

00 0001 0000 0000 0000 0000 0000 0111

0x0100007

Jump Format Instructions



- Assembly Language: **j b #address 0x0040001C**
- Machine Language: **0000 1000 0001 0000 0000 0000 0000 0111**

Pseudo Operations

- An **pseudo operation** (or **pseudo op**) is an instruction/operation that does not correspond to any machine language instruction.
 - Pseudo ops exist for convenience, not out of necessity
- The assembler translates the pseudo op into one or more real instructions.
 - Each assembler has its own pseudo operations
 - The pseudo ops shown in the next few slides are specific to the MARS assembler, but can be found in other MIPS assemblers as well

Pseudo Operations

- The load immediate (**li**) instruction is a pseudo op.

li \$t0, 5

- The MARS assembler converts this to an **addi** instruction

addi \$t0, \$0, 5

- Adds the constant with 0 and assigns the result to the specified register

li \$t0, 5



addi \$t0, \$0, 5

Pseudo Operations

- The load address (**la**) instruction is a pseudo op.

la \$t0, 8(\$t1)

- The MARS assembler converts this to an **addi** instruction

addi \$t0, \$t1, 8

- Adds the constant to the address in \$t1 and assigns the result to \$t0

la \$t0, 8(\$t1)



addi \$t0, \$t1, 8

Pseudo Operations

- The **move** instruction is a pseudo op.

move \$t0, \$t1

- The MARS assembler converts this to an **add** instruction

add \$t0, \$0, \$t1

- Adds the value in \$t1 with 0 and assigns the result to \$t0

move \$t0, \$t1



add \$t0, \$0, \$t1

Pseudo Operations

- The **not** instruction is a pseudo op.

not \$t0, \$t1

- The MARS assembler converts this to a **nor** instruction

nor \$t0, \$t1, \$0

- nor's the value in \$t1 with 0 and assigns the result to \$t0

not \$t0, \$t1



nor \$t0, \$t1, \$0

Pseudo Operations

- The branch if less than (**blt**) instruction is a pseudo op.

blt \$t0, \$t1, label

- The MARS assembler converts this to two instructions:

slt \$at, \$t0, \$t1

bne \$at, \$0, label

- \$at is set to 1 if \$t1 is less than \$t0
- If \$at is not equal to 0, then \$t1 was less than \$t0 and the branch occurs

blt \$t0, \$t1, label



slt \$at, \$t0, \$t1
bne \$at, \$0, label

Pseudo Operations

- The branch if less than or equal (**ble**) instruction is a pseudo op.

ble \$t0, \$t1, label

- The MARS assembler converts this to two instructions:

slt \$at, \$t1, \$t0

beq \$at, \$0, label

- \$at is set to 1 if \$t0 is less than \$t1
- If \$at is equal to 0, then \$t1 was less than or equal to \$t0 and the branch occurs

ble \$t0, \$t1, label



slt \$at, \$t1, \$t0
beq \$at, \$0, label

Pseudo Operations

- The branch if greater than (**bgt**) instruction is a pseudo op.

bgt \$t0, \$t1, label

- The MARS assembler converts this to two instructions:

slt \$at, \$t1, \$t0

bne \$at, \$0, label

- \$at is set to 1 if \$t0 is less than \$t1
- If \$at is not equal to 0, then \$t1 was greater than \$t0 and the branch occurs

bgt \$t0, \$t1, label



slt \$at, \$t1, \$t0
bne \$at, \$0, label

Pseudo Operations

- The branch if greater than or equal (**bge**) instruction is a pseudo op.

bge \$t0, \$t1, label

- The MARS assembler converts this to two instructions:

slt \$at, \$t0, \$t1

beq \$at, \$0, label

- \$at is set to 1 if \$t1 is less than \$t0
- If \$at is equal to 0, then \$t1 was greater than or equal to \$t0 and the branch occurs

bge \$t0, \$t1, label



slt \$at, \$t0, \$t1

beq \$at, \$0, label